

08. Manejo de texto en R

Ciencia de datos para la toma de decisiones II

Jorge

Juvenal

Campos Ferreira

✉ juvenal.campos@tec.mx

Programa de la sesión de hoy

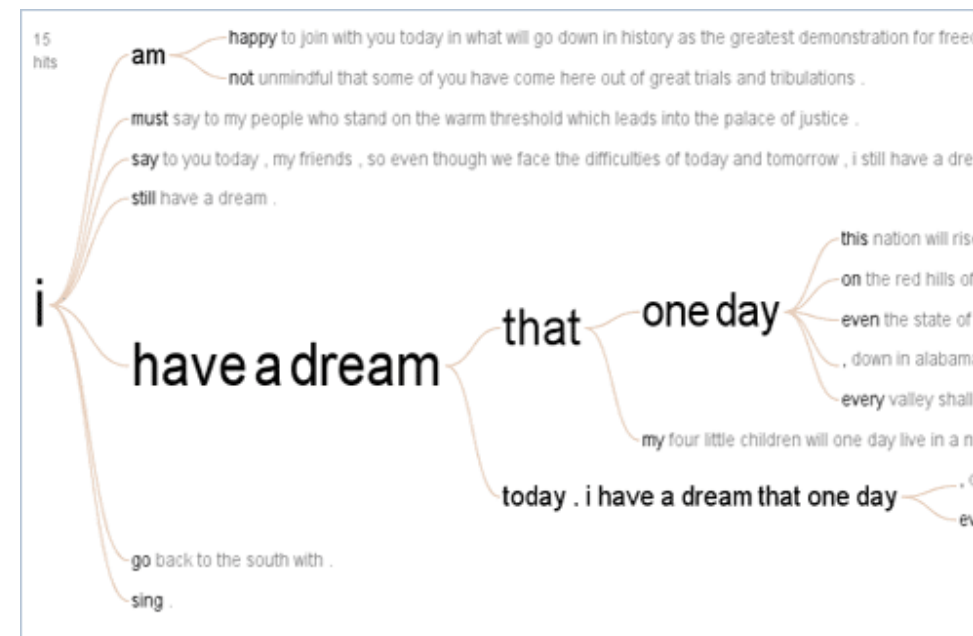
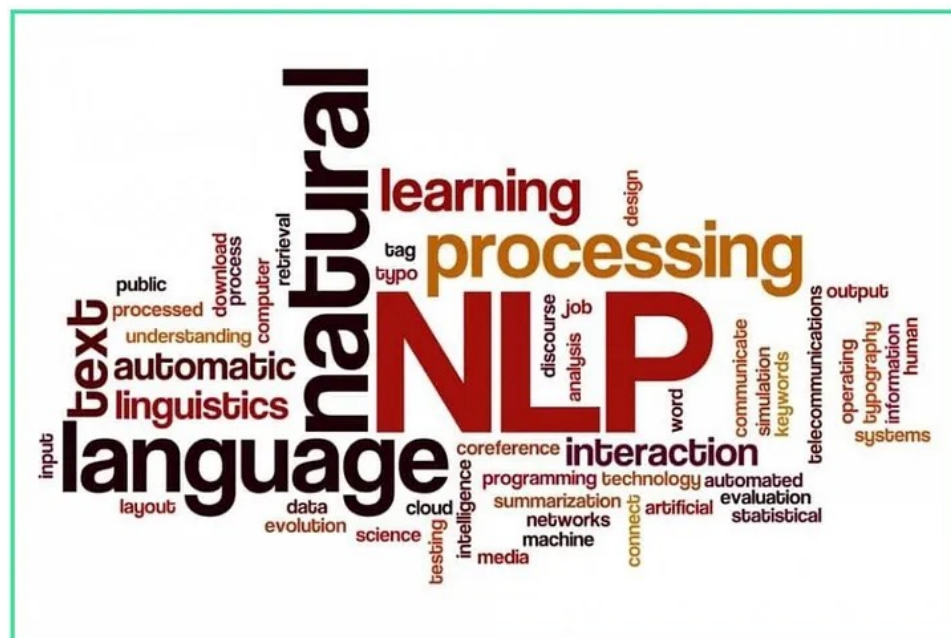
- Texto como datos
- Ejemplo de análisis de texto
- Fundamentos: funciones de *stringr*
- Ejercicio con expresiones regulares

Texto como datos

Es un enfoque para tratar al lenguaje (documentos, posts, discursos, leyes, noticias, reseñas) como datos cuantificables, de modo que se pueda analizar con métodos estadísticos o de *Machine Learning*.

La diferencia con los datos numéricos es:

- 1) Alta dimensionalidad.
- 2) Requiere métodos especializados para su análisis.



```
# Cargar librerías necesarias
library(tm)      # Para procesamiento de texto
library(Matrix)  # Para matrices dispersas
library(ggplot2) # Para visualizaciones
library(tidyr)   # Para manipulación de datos

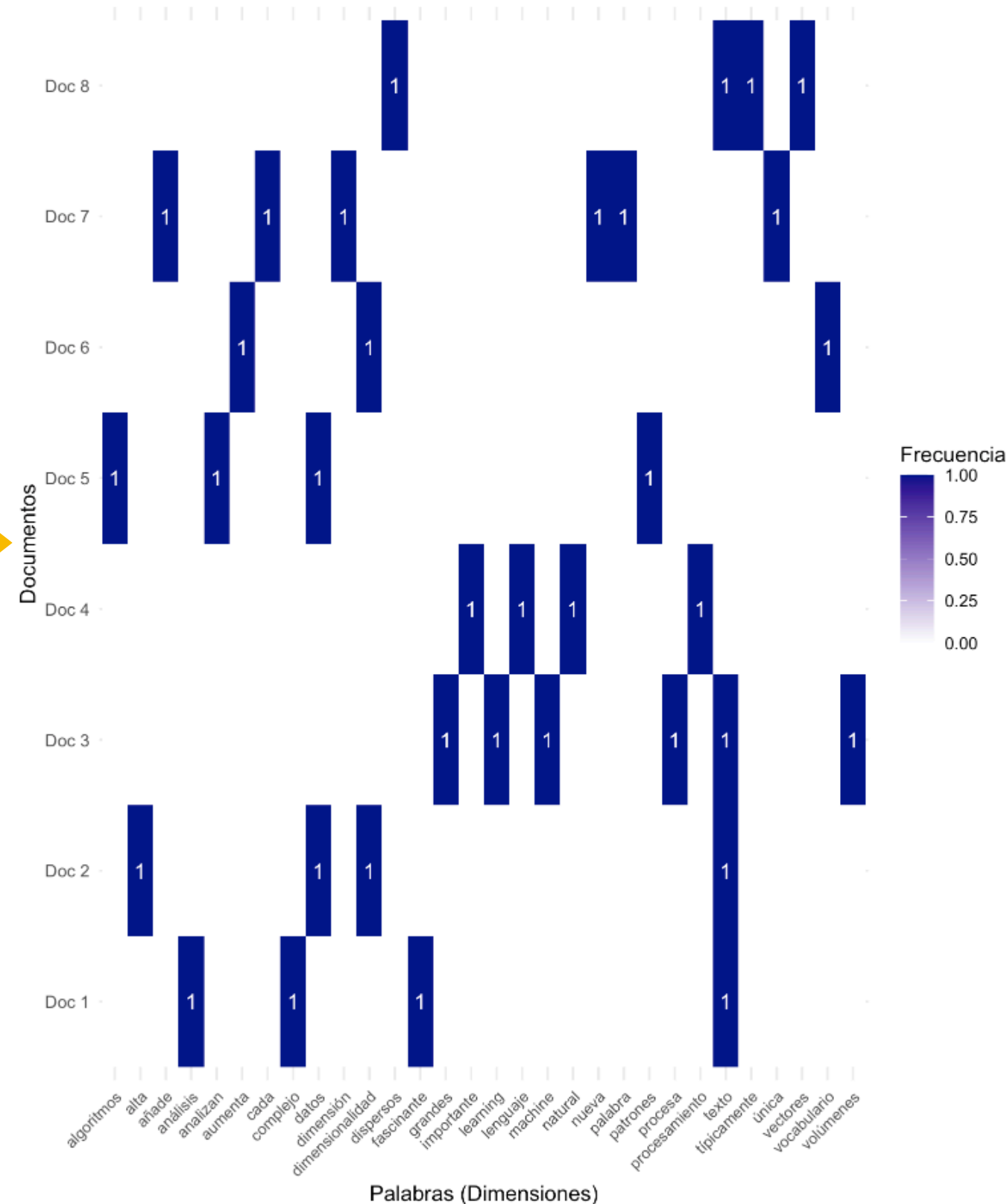
# Configurar semilla para reproducibilidad
set.seed(123)

# 1. CREAR UN CORPUS DE EJEMPLO
# -----
documentos <- c(
  "El análisis de texto es fascinante y complejo",
  "Los datos de texto tienen alta dimensionalidad",
  "Machine learning procesa grandes volúmenes de texto",
  "El procesamiento del lenguaje natural es importante",
  "Los algoritmos analizan patrones en los datos",
  "La dimensionalidad aumenta con el vocabulario",
  "Cada palabra única añade una nueva dimensión",
  "Los vectores de texto son típicamente dispersos"
)

corpus <- Corpus(VectorSource(documentos))
corpus <- tm_map(corpus, content_transformer(tolower))
corpus <- tm_map(corpus, removePunctuation)
corpus <- tm_map(corpus, removeWords, stopwords("spanish"))
corpus <- tm_map(corpus, stripWhitespace)
```



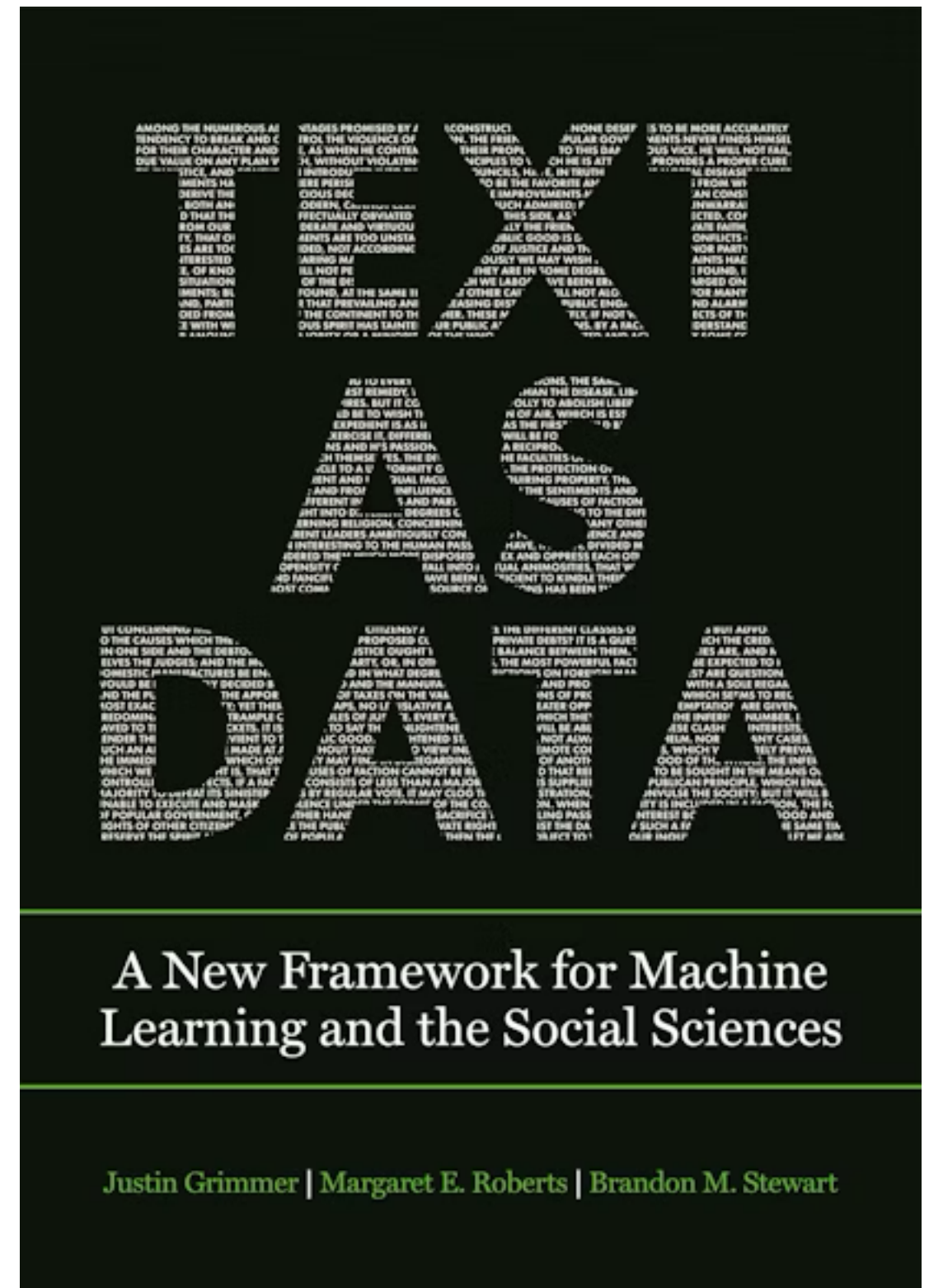
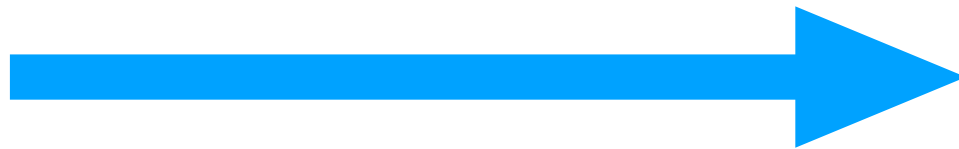
Matriz Documento-Término: Visualización de Alta Dimensionalidad
Cada fila es un documento, cada columna una palabra única (30 dimensiones)



Etapas del análisis de texto

Basado en el libro de “Text as data” de Grimmer, Roberts y Stewart”

El libro se enfoca principalmente en los pasos del proceso de análisis, en vez de los algoritmos de análisis.



Etapas del análisis de texto

Descubrimiento (Discovery)

El **descubrimiento** es el proceso de **crear nuevas conceptualizaciones o formas de organizar los datos**. Es la **etapa inicial** de un proyecto de investigación en la que el investigador se centra en desarrollar la pregunta de investigación.

En esta fase, **se decide qué se quiere medir a partir de los datos y cuál será el objetivo de la inferencia**. El análisis de texto ayuda en este proceso al sugerir nuevas formas de organizar una colección de documentos, como identificando grupos de textos similares (*clusters*) o características subyacentes en la colección.

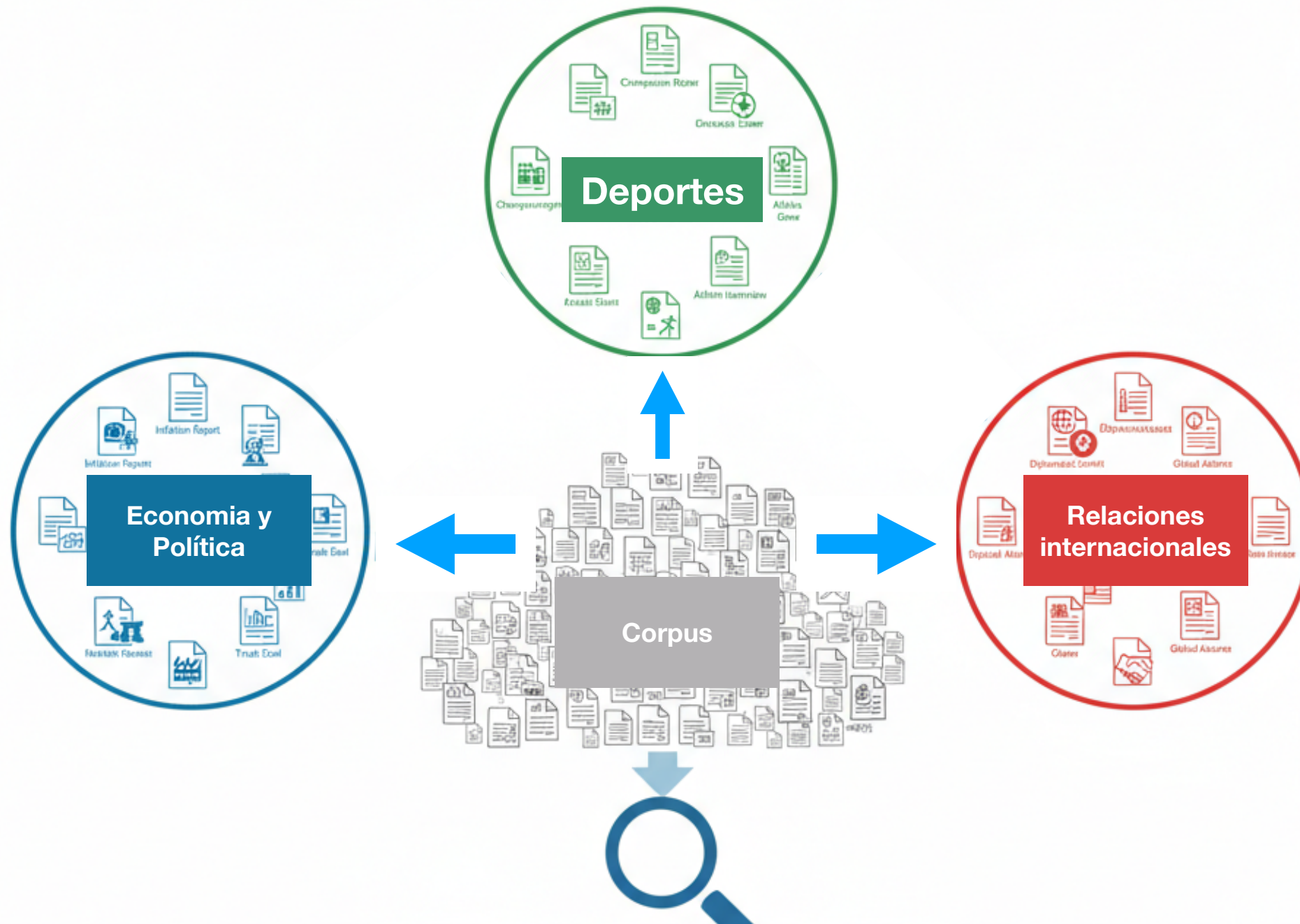
La utilidad de una **organización** no se juzga por si es "correcta" o no, sino por si es útil para responder a una pregunta de investigación.

Organización

Una **organización** es una estructura o esquema de clasificación que se aplica a un conjunto de documentos de texto para agruparlos, ordenarlos o describirlos de forma sistemática.

Esta organización puede ser generada por un investigador de manera manual, descubierta por un algoritmo o una combinación de ambos.

Su objetivo no es ser una representación “verdadera” de la realidad, sino ser útil para la investigación científica al simplificar la complejidad inherente a los datos de texto de alta dimensionalidad.



Propósito de las organizaciones: Hacer que el corpus sea útil para responder las preguntas de investigación. No necesariamente que sea una clasificación correcta.

Etapas del análisis de texto

Medición (Measurement)

La **medición** es el proceso de **conectar los conceptos con los datos**, lo que permite describir la prevalencia de esos conceptos en el mundo real.

La meta es crear una simplificación del fenómeno del mundo real que permita al analista describir con precisión un proceso o probar las implicaciones observables de una teoría.

Los métodos de medición varían desde el simple conteo de palabras clave hasta el uso de aprendizaje supervisado para clasificar documentos.

La validación es un paso crucial en la medición para asegurar que la medida refleja el concepto que se pretende capturar.

Etapas del análisis de texto

Predicción (Prediction)

La **predicción** es la tarea de estimar el valor de un resultado para una unidad que se encuentra fuera de la población utilizada para entrenar el modelo.

El objetivo es pronosticar lo que sucederá en el futuro, o lo que está sucediendo ahora (nowcasting), utilizando la información disponible en el presente.

A diferencia de la inferencia causal, las características utilizadas para la predicción no tienen que ser necesariamente la causa del resultado. Lo que importa es la precisión del pronóstico.

Un **ejemplo** es el uso de datos de redes sociales para predecir los resultados de encuestas de opinión.

Etapas del análisis de texto

Inferencia Causal (Causal Inference)

La **inferencia causal** se enfoca en **determinar cómo cambiarían los resultados si se interviniera en el mundo**. La meta es estimar lo que **habría sucedido** en un escenario contrafactual (no observado) si la intervención de interés hubiera sido diferente.

Este tipo de inferencia se basa en un diseño de investigación cuidadoso para garantizar que la comparación observada sea una estimación plausible del estado contrafactual.

El libro subraya que el uso de datos de texto no exime a los investigadores de los desafíos fundamentales de la inferencia causal, como el sesgo por variables de confusión o la falta de independencia entre las observaciones

Ejemplos de análisis de texto

Investigadores del Centro de Investigación en Computación (CIC-IPN) aplicaron técnicas de **minería de texto**, **procesamiento de lenguaje natural** y algoritmos de **aprendizaje automático** para realizar un **análisis de sentimiento** de tuits relacionados con el TLCAN.

Probaron múltiples algoritmos de clasificación de texto (Naive Bayes, árboles de decisión, SVM, etc.), encontrando que la combinación de varios modelos brindó el mejor desempeño (aprox. 88% de exactitud) en la detección automática de sentimiento positivo/negativo en español

Aguilar, D., Sidorov, G., & Batyrshin, I. (2018). *Caso de estudio de análisis de sentimientos en Twitter: Tratado de Libre Comercio de América del Norte*. *Research in Computing Science*, 147(5), 357–365. https://rccs.cic.ipn.mx/2018_147_5/Caso%20de%20estudio%20de%20análisis%20de%20sentimientos%20en%20Twitter%20Tratado%20de%20libre%20comercio%20de%20America.pdf

Ejemplos de análisis de texto

Se analizó la influencia de la narrativa económica de sus informes sobre las expectativas de inflación y crecimiento. Para ello, los investigadores utilizaron **modelado de temas** (Asignación Latente de Dirichlet, *LDA*) sobre el texto de los resúmenes de Informes Trimestrales del banco central, identificando automáticamente 26 **temas** recurrentes en esos documentos.

A cada tema se le midió el “**tono**” y su cambio a lo largo del tiempo, incorporando esas variables cualitativas en un modelo econométrico. Los resultados sugieren que estas **señales narrativas** tienen efecto significativo en las expectativas de los analistas privados, complementando la influencia de las proyecciones cuantitativas

Antón, A., Bazdresch, S., & Lelo de Larrea, A. (2023). *Influencia de las proyecciones del banco central y la narrativa económica sobre las expectativas de los pronosticadores profesionales: Evidencia de México* (Documento de investigación del Banco de México No. 2023-21). Banco de México. <https://www.banxico.org.mx/publicaciones-y-prensa/documentos-de-investigacion-del-banco-de-mexico/resumenes-ejecutivos/%7BC561B5CD-1917-E6BD-FD36-1E9A47B96543%7D.pdf>

Ejemplos de análisis de texto

Estudio reciente el uso de **NLP** para analizar grandes volúmenes de texto, aplicando **reconocimiento de entidades** y **modelado de temas** a documentos y **quejas de consumidores** para identificar tendencias emergentes

Cabello Ávila, C., & Koike Quintanar, S. A. (2024). *Potenciales riesgos y beneficios en materia de competencia económica en el entorno de la inteligencia artificial*. Instituto Federal de Telecomunicaciones, Centro de Estudios. <https://centrodeestudios.ift.org.mx/admin/files/estudios/1737564362.pdf>

Ejemplos de análisis de texto

El estudio *“Análisis con minería de datos del Marco Curricular y Plan de Estudios 2022 de la Educación Básica Mexicana”* examinó el nuevo marco curricular de educación básica y lo comparó con sus antecesores (Plan 2011 y Aprendizajes Clave 2017).

Para ello se construyó un corpus con los textos de los tres planes educativos y se aplicaron técnicas de minería de texto en R (usando paquetes como *Quanteda*, *tidyverse* y hasta *word2vec*).

Mediante métodos no supervisados (por ejemplo, modelado de temas LDA) y supervisados, identificaron diferencias semánticas y de énfasis entre los documentos. **Los resultados revelan cambios importantes en el discurso educativo: el plan 2011 se enfocaba más en disciplina, el de 2017 en competencias y eficiencia, mientras que el marco 2022 resalta el desarrollo comunitario, la inclusión y el bienestar de poblaciones vulnerables.**

Duarte V., 2022. Universidad Vasco de Quiroga. Análisis con minería de datos del Marco Curricular y Plan de Estudios 2022 de la Educación Básica Mexicana. Consultado en: <https://dialnet.unirioja.es/servlet/articulo?codigo=9869691#:~:text=mediante%20el%20procesamiento%20de%20texto,de%20la%20población%20en%20desigualdad>

Librería *stringr*



Librería *stringr*



- La librería *stringr* es la librería del tidyverse que se usa para trabajar con texto (objetos carácter).
- **Permite realizar:**
 - o Concatenación de cadenas con *str_c()*
 - o Obtención de longitud, con *str_length()*
 - o Extracción de subcadenas, con *str_extract()* y *str_extract_all()*
 - o Eliminación de subcadenas, con *str_remove()*
 - o Reemplazo de patrones, con *str_replace()* y *str_replace_all()*

Entre otras funciones

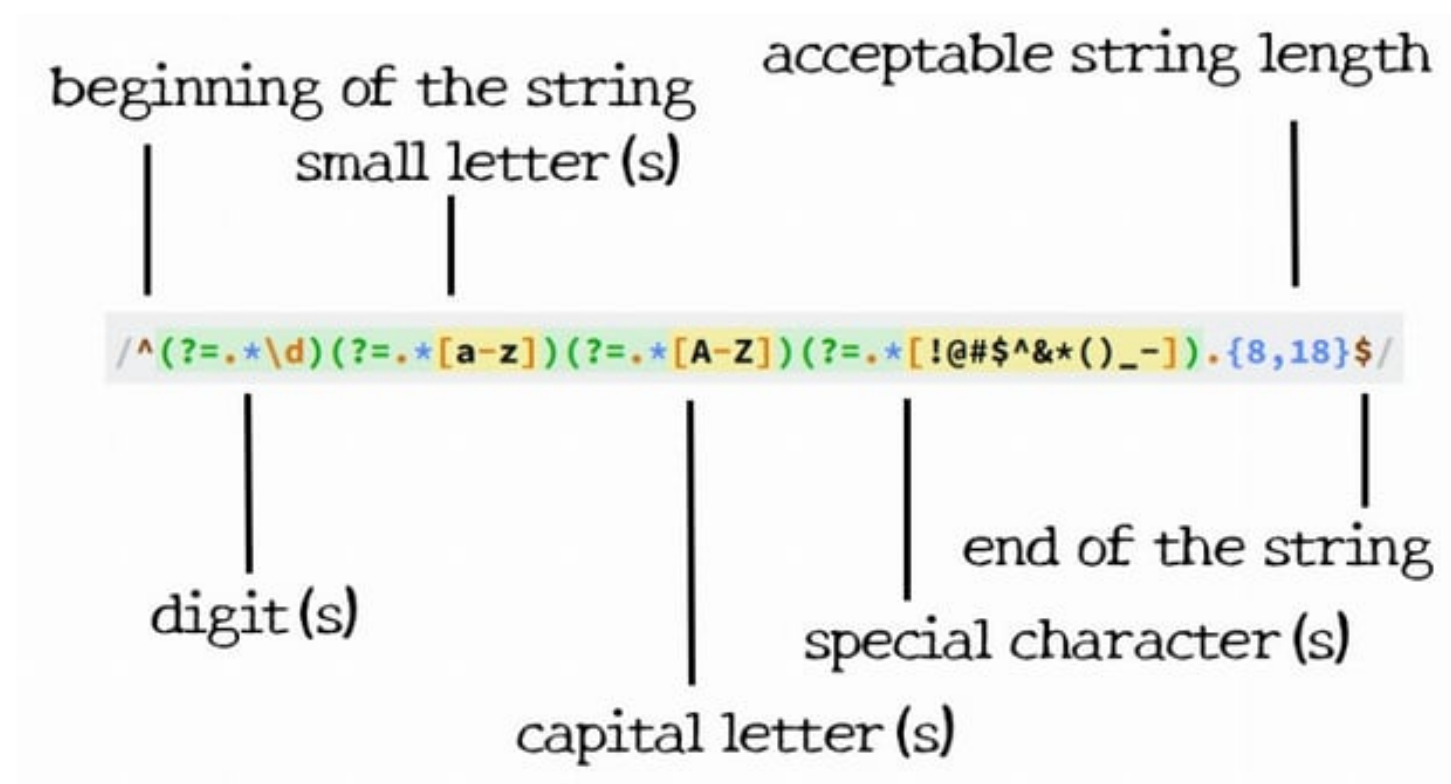
Estas funciones se utilizan en conjunto con las expresiones regulares.

Cheatsheet: <https://rstudio.github.io/cheatsheets/html/strings.html>

Expresiones regulares

Son secuencias de caracteres que definen patrones de búsqueda. Son una herramienta poderosa y flexible para el preprocesamiento de texto.

- Usadas con las funciones de *stringr* se pueden realizar búsquedas, extracciones y reemplazos con patrones complejos.



REGEX CHEATSHEET

EVERYTHING YOU NEED TO KNOW WHEN WORKING WITH REGULAR EXPRESSIONS (POSIX STANDARDS)



Repetitions of a pattern

*	at least 0 times	abc* "ab" followed by zero or more "c"
+	at least 1 time	abc+ "ab" followed by one or more "c"
?	at most 1 time	abc? "ab" followed by zero or one "c"
{n}	exactly n times	abc{2} "ab" followed by 2 "c"
{n,}	at least n times	abc{3,} "ab" followed by at least 3 "c"
{n, m}	at least n times, at most m times	abc{3,5} "ab" followed by 3 up to 5 "c"

Positions of a pattern

^	the start of the string	^starts string starting with "starts"
\$	the end of the string	ends\$ string ending with "ends"
\b	empty string at edge of a word	\\bab matches "ab" in "abc" but not in "dabc"
\B	empty string not at edge of a word	\\Bab matches "ab" in "dabc" but not in "abc"

Character classes I

\d	digits	3\\d{10} 11-digit strings starting with "3"
\D	non-digits	a\\D matches "ab" but not "a1"
\w	word characters (incl. underscore)	ab\\w matches "abc" & "ab_" but not "ab."
\W	non-word characters	ab\\W matches "ab." but not "abc"
\s	space	ab\\sc matches "ab c" but not "abc"
\S	non-space	a\\Sc matches "abc" but not "a c"

Character classes II

[[:digit:]] or [0-9]	digits	ab[[:digit:]]+ matches "ab3" & "ab221"
[[:lower:]] or [a-z]	lowercase letters	ab[[:lower:]] matches "abc" but not "abC"
[[:upper:]] or [A-Z]	uppercase letters	ab[[:upper:]] matches "abC" but not "abc"
[[:alpha:]] or [A-z]	alphabetic characters	ab[[:alpha:]] matches "abc" & "abC"
[[:alnum:]]	alphanumeric characters	ab[[:alnum:]] matches "ab3", "abc" & "abC"
[[:xdigit:]]	hexadecimal characters (base 16)	ab[[:xdigit:]] matches "abf" but not "abg"
[[:blank:]]	blank characters (space & tab only)	ab[[:blank:]]c matches "ab c" & "ab\tc" but not "abc"
[[:space:]]	space characters*	ab[[:space:]]c matches "ab c", "ab\tc" & "ab\nc"
[[:punct:]]	punctuation characters	ab[[:punct:]] matches "ab!" and "ab@"
[[:graph:]]	graphical characters	ab[[:graph:]]+ matches "ab!", "abc!" & "ab3@"
[[:print:]]	printable characters	ab[[:print:]]+ matches "ab 3", "ab c!" & "abc"

*Space characters are tab (\t), newline (\n), vertical tab (\v), carriage return (\r), form feed (\f), and space

Others

.	any single character	abc. "abc" followed by any single character
[...]	character list	ab[0-9]+ matches "ab3" & "ab221"
[^...]	inverted character list	ab[^0-9]+ matches "abcd", but not "ab3d"
\	escape character	ab* "ab" followed by ""
	"or" operator	a(b c) "a" followed by "b" or "c"
(...)	grouping	a(bcd) "a" followed by sequence "bcd"
\\N	backreference to Nth capture group	(co)\\1 matches "coconut" but not "count"

Ejercicio

1. Abra el ejercicio de la carpeta de “*ejercicios prácticos*” de la sesión de hoy y resuelva los ejercicios marcados como comentarios del script “02_ejercicio_regex.R” utilizando las funciones de manejo de texto (*stringr*) y expresiones regulares que considere necesarias.

Conceptos básicos

Documento: El documento es la unidad fundamental de análisis.

Puede ser cualquier cosa, desde un único mensaje de Twitter hasta un informe anual de una empresa o un discurso de un político.

La forma de definir un documento depende del objetivo de la investigación. Por ejemplo, para la detección de spam, un documento podría ser un correo electrónico individual. Para el análisis de precios de acciones a partir de noticias, un documento podría ser el texto de las noticias de un día en particular.

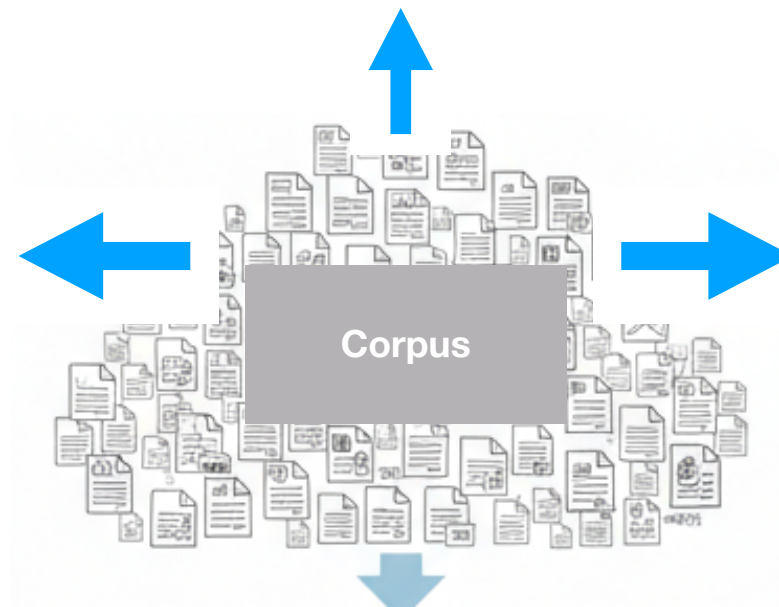
Conceptos básicos

La **codificación de caracteres** es la forma en que las letras, símbolos y textos se representan internamente en la computadora mediante números.

Por ejemplo, **UTF-8** es un estándar de codificación que asigna a cada carácter una secuencia de uno a cuatro bytes. UTF-8 puede representar prácticamente cualquier símbolo de casi todos los idiomas (acentos, caracteres chinos, emojis, etc.) evitando errores donde aparecen caracteres extraños por una mala interpretación de los datos, lo que lo hace ideal para el intercambio de información mundial.

Conceptos básicos

Corpus: Un corpus es una colección de documentos. Es el conjunto de texto que se va a analizar.



Conceptos básicos

Token: Un token es una unidad de lenguaje, como una palabra o una frase, que se cuenta o se analiza. La representación más simple del texto, el modelo de "bolsa de palabras" (bag-of-words), utiliza tokens y su frecuencia, ignorando el orden en que aparecen.

Por ejemplo, en la frase "Good night, good night!" el token "good" aparece dos veces y "night" también.

Conceptos básicos

La **tokenización** es el proceso de dividir un texto en unidades más pequeñas llamadas *tokens*.

Comúnmente, cada token es una palabra, pero según la necesidad también pueden ser oraciones, caracteres individuales o incluso sub-palabras.

El objetivo es facilitar el análisis: en lugar de procesar un párrafo entero como una cadena larga, se le separa en piezas manejables.

Por ejemplo, al tokenizar "*Hola mundo, ¿cómo estás?*" podríamos obtener los tokens: ["Hola", "mundo", ",", "¿", "cómo", "estás", "?"]. Una vez tokenizado, es más sencillo contar frecuencia de palabras, filtrarlas (como eliminar *stopwords*), o alimentar esos tokens a algoritmos de NLP. *Ilustración sugerida:* La imagen de una frase que se “rompe” en fichas o bloques individuales (cada palabra por separado).

Conceptos básicos

Vocabulario: Es el conjunto de todos los tokens únicos en un corpus.

El tamaño del vocabulario puede ser enorme, lo que contribuye a la alta dimensionalidad del texto.

Conceptos básicos

La **lematización** es una técnica para reducir las palabras a su forma base o *lema*, que es la forma que encontraríamos en el diccionario.

A diferencia de simplemente recortar terminaciones, la lematización utiliza reglas lingüísticas (y a veces diccionarios) para mapear, por ejemplo, un verbo conjugado a su infinitivo, o un sustantivo plural a su singular. Por ello, *corriendo*, *corrí* y *corre* se lematizarían todas como "*correr*".

El propósito es tratar variantes de una palabra como la misma entidad durante el análisis, lo que mejora la consistencia en tareas como comparación de documentos o indexación.

Ejemplo: Un lematizador procesando la frase "Los niños **jugaban** en los parques" transformaría "*jugaban*" -> "*jugar*" y "*parques*" -> "*parque*".

Conceptos básicos

El **stemming** (derivación) también reduce las palabras a una raíz común, pero utilizando métodos más simples y mecánicos que la lematización.

En esencia, un algoritmo *stemmer* corta prefijos o sufijos de las palabras para eliminar *morfemas flexivos**. Por ejemplo, aplicando stemming en inglés: "connections", "connected", "connecting" todas podrían truncarse al stem "connect"; en español, "jugar", "jugando", "jugarán" podrían reducirse a "jug" o "jug".

La raíz obtenida no siempre es una palabra completa con significado (a veces son fragmentos), pero suele bastar para agrupar textos que hablan de lo mismo. El stemming es rápido y útil para reducir dimensionalidad (menos formas distintas), aunque sacrifica precisión porque no distingue contextos ni excepciones gramaticales.

Ejemplo: Con un **Porter stemmer** (algoritmo clásico de stemming), las palabras "corriendo", "corre" y "corrí" se recortan todas a "corr".

* Los **morfemas flexivos** son las **terminaciones** que se añaden a una palabra para expresar rasgos **gramaticales** (sin crear una palabra nueva). Señalan cosas como **género** y **número** en sustantivos/adjetivos, y **persona**, **número**, **tiempo**, **modo** y **aspecto** en verbos.

Conceptos básicos

Un **diccionario de normalización** es un recurso (generalmente una tabla de equivalencias) usado para reemplazar variantes textuales por una forma estándar común.

Contiene entradas de texto “no normalizado” y su correspondiente forma *normalizada*. Esto sirve para unificar escritura de palabras que pueden aparecer de distintas maneras. Por ejemplo, un diccionario podría mapear "EEUU", "E.E.U.U." o "Estados unidos" -> "Estados Unidos"; o en chats, convertir "xq" -> "porque", "tl;dr" -> "resumen" .

Al aplicar este diccionario a un texto, todas esas variantes se sustituyen por la forma elegida, facilitando análisis posteriores (p. ej., conteos o comparaciones) donde ya no importan las diferencias de ortografía o formato. En otros casos, estos diccionarios también corrigen errores comunes (como "*tecnologia*" -> "*tecnología*" con tilde) o expanden abreviaturas ("*av*" -> "*avenida*").

Ejemplo: En análisis de redes sociales, definir un diccionario para normalizar emoticones a palabras (":)") -> "feliz") o jerga como "tqm" -> "te quiero mucho".

Conceptos básicos

OCR (Reconocimiento Óptico de Caracteres, por sus siglas en inglés) es la tecnología que permite convertir una imagen que contiene texto en texto digital editable.

Básicamente, un sistema OCR “lee” una imagen escaneada de una página o una foto de un cartel y **detecta las letras** impresas para producir un documento de texto plano con ese mismo contenido.

Esto es útil para digitalizar documentos en papel (libros, facturas, artículos antiguos) y luego poder buscar palabras en ellos o editar el contenido. Los motores OCR modernos utilizan algoritmos de visión por computadora y aprendizaje automático entrenados con muchas fuentes y escrituras para reconocer caracteres con gran precisión, incluso si la imagen tiene ruido o las letras estilos diferentes.

Ejemplo: Si escaneamos una carta manuscrita antigua, un OCR intentará identificar cada carácter escrito a mano y generar un archivo de texto; o cuando depositas un cheque, el banco usa OCR para leer los números.

Conceptos básicos

El **data crawling** (o *web crawling*) es un proceso automatizado de **navegación sistemática por la web** para recopilar datos de múltiples páginas.

Un programa llamado *crawler* o *araña web* comienza con una URL inicial, descarga ese contenido, extrae de allí nuevos enlaces y los visita, y así sucesivamente, explorando buena parte de la red (o de un sitio objetivo). Mientras recorre páginas, va guardando información relevante de cada una: puede ser el HTML completo, texto específico, meta-datos, etc. Los datos recolectados luego pueden indexarse para búsquedas (como hacen los motores tipo Google) o usarse para análisis (por ejemplo, armar un corpus de noticias).

Ejemplo: Una aplicación de *data crawling* podría empezar en la página de Wikipedia sobre “*Inteligencia Artificial*”, extraer los enlaces de ese artículo, visitar cada uno (ej. *Aprendizaje automático*, *Redes neuronales*, etc.), seguir recursivamente... y así construir un dataset grande de artículos de IA.

Conceptos básicos

Una **DTM (Document-Term Matrix)** o **DFM (Document-Feature Matrix)** es una matriz que describe cuantitativamente un conjunto de documentos mediante la frecuencia de aparición de términos (palabras) en cada documento.

En dicha matriz, cada **fila** representa un documento distinto, cada **columna** representa un término del vocabulario, y cada celda (*fila*, *columna*) contiene típicamente el número de veces que el término de esa columna aparece en el documento de esa fila. Es, en esencia, un formato de tabla para el modelo *bolsa de palabras*. Por ejemplo, si tenemos 3 documentos y nuestro vocabulario consiste en 4 palabras, la DTM podría ser:

Doc	palabra1	palabra2	palabra3	palabra4
Doc1	2	0	1	0
Doc2	0	1	3	1
Doc3	1	1	0	2

Conceptos básicos

Un **n-grama** es una secuencia contigua de n elementos en un texto.

En PLN (NLP), normalmente esos elementos son palabras; así, un **unigrama** (1-grama) es una sola palabra, un **bigrama** (2-grama) son dos palabras seguidas, un **trigrama** tres palabras seguidas, y así sucesivamente.

Por ejemplo, para la frase *"el gato negro come"*: sus bigramas serían ["el gato", "gato negro", "negro come"] y sus trigramas ["el gato negro", "gato negro come"].

Los n-gramas capturan algo de contexto que se pierde si consideramos palabras aisladas; por ejemplo un bigrama puede representar una expresión fija (*"realidad virtual"*, *"mesa redonda"*) o ayudar en modelos de lenguaje a predecir la siguiente palabra basándose en las anteriores.

Ejemplo: En modelado de texto, un analizador de sentimiento podría notar que el trigrama *"no me gusta"* tiene connotación negativa fuerte. Muchas técnicas computan frecuencias de n-gramas para detectar qué combinaciones son las más típicas de un corpus.

Conceptos básicos

- **NLP** significa *Natural Language Processing*, o **Procesamiento del Lenguaje Natural (PLN)** en español. Es el campo de la inteligencia artificial enfocado en lograr que las máquinas puedan **entender, interpretar y generar** lenguaje humano de manera útil.
- El **NLP** Incluye una amplia gama de tareas: *entender* instrucciones (como hace un asistente tipo Siri), *traducir* automáticamente de un idioma a otro, *resumir* documentos largos, *analizar sentimientos* en tweets, *reconocer entidades* (como nombres propios en texto), *corregir ortografía*, entre muchas más.
- Para ello, NLP combina técnicas de lingüística computacional (conocimiento de gramática, sintaxis, semántica) con algoritmos de machine learning y estadística.
- En los últimos años, el PLN ha avanzado enormemente gracias a modelos de aprendizaje profundo entrenados con grandes cantidades de datos textuales.
- **Ejemplo:** Un sistema de NLP típico es un chatbot de soporte: recibe preguntas de usuarios en lenguaje natural ("¿Cómo recupero mi contraseña?"), las interpreta (entiende que es una pregunta sobre *recuperación de contraseña*) y genera una respuesta apropiada en lenguaje natural.

Conceptos básicos

TF-IDF es un método de ponderación de términos muy utilizado en recuperación de información y minería de texto. Sus siglas significan *Term Frequency - Inverse Document Frequency*, o sea, *Frecuencia de término - Frecuencia inversa de documentos*.

La idea es asignar a cada palabra en un documento un peso que refleje qué tan importante es esa palabra en ese documento **y** a la vez penalizar las palabras demasiado comunes en todos los documentos. Se calcula multiplicando dos componentes:

- **TF** (term frequency): normalmente el conteo de la palabra en el documento (o una versión normalizada).
- **IDF** (inverse document frequency): un factor que disminuye el peso si la palabra aparece en muchos documentos del corpus. Clásicamente $IDF = \log(N/n)$, donde N es el número total de documentos y n el número de documentos que contienen esa palabra.

Conceptos básicos

Así, palabras frecuentes en un documento pero raras en el resto del corpus obtienen un TF-IDF alto, señal de que son representativas de ese documento en particular. En cambio, palabras comunes (ej. "de", "la", "y") tendrán TF-IDF muy bajo aunque aparezcan mucho, porque carecen de poder discriminante.

Ejemplo: En un corpus de noticias, el término "elecciones" tendrá TF-IDF alto en un artículo político si aparece muchas veces allí y pocos artículos más lo mencionan; pero la palabra "el" tendrá TF-IDF cercano a cero en todos los documentos.

Conceptos básicos

El **modelado supervisado** se refiere a construir modelos de *machine learning* utilizando datos de entrenamiento **etiquetados**, es decir, donde para cada ejemplo conocemos la respuesta esperada.

El objetivo es que el modelo aprenda una función que, dadas las características de entrada (en nuestro caso, texto preprocesado), prediga correctamente la etiqueta de salida. En PLN (NLP), ejemplos típicos de modelado supervisado incluyen:

- **Clasificación de texto:** entrenar un modelo con correos marcados como *spam* o *no spam* para que aprenda a clasificar nuevos correos.
- **Análisis de sentimiento:** un dataset de reseñas de películas con su calificación (*positiva/negativa*) para predecir el sentimiento de nuevas reseñas.

Conceptos básicos

- **Reconocimiento de idioma:** con frases etiquetadas por idioma, entrenar un clasificador que dada una frase diga si está en español, inglés, etc.
El proceso usual es: se representa el texto en forma numérica (por ejemplo con atributos como palabras frecuentes, embeddings, etc.), se entrena un algoritmo (p. ej. regresión logística, árboles, redes neuronales) minimizando el error en predecir las etiquetas conocidas, y luego se evalúa el desempeño en datos nuevos.
- *Ejemplo:* Un ejemplo de uso sería entrenar un modelo **SVM** con titulares de noticias etiquetados en categorías (*deportes, política, economía*, etc.). Una vez entrenado con suficientes ejemplos, al darle un titular nuevo (sin etiqueta) el modelo dirá, por ejemplo, "esto suena a noticia de *deportes* con 95% de confianza" (**predicción**).

Conceptos básicos

- Los **embeddings** son representaciones vectoriales densas de texto en un espacio matemático de muchas dimensiones, diseñadas para capturar el significado semántico.
- En lugar de representar una palabra de forma *one-hot* (un vector gigante con un 1 en la posición de esa palabra y 0 en las demás), un embedding es un vector más pequeño (digamos de 100 o 300 dimensiones) con valores continuos, entrenado de tal manera que palabras con significados parecidos tengan vectores cercanos entre sí en ese espacio.
- Por ejemplo, un buen modelo de embeddings situaría a "*rey*" cerca de "*reina*" y más lejos de "*tortilla*". Estos vectores se aprenden *leyendo* enormes cantidades de texto y ajustándose para predecir contextos de palabras (modelo tipo Word2Vec, GloVe) o mediante capas internas de redes neuronales (como las de BERT).
- Una propiedad famosa de algunos embeddings es que permiten operar aritméticamente con significado: $\text{vector}(\text{"Rey"}) - \text{vector}(\text{"Hombre"}) + \text{vector}(\text{"Mujer"}) \approx \text{vector}(\text{"Reina"})$, ilustrando que la relación *Rey: Hombre* es análoga a *Reina: Mujer*.

Conceptos básicos

- El **modelado de temas** (*Topic Modeling*) es un conjunto de técnicas **no supervisadas** para descubrir automáticamente los *temas latentes* presentes en una colección de documentos.
- A diferencia de un modelo supervisado, aquí no tenemos etiquetas predefinidas; el algoritmo infiere patrones de co-ocurrencia de palabras y agrupa palabras que tienden a aparecer juntas, definiendo así un “tema”. Cada **tema** se suele representar por un conjunto de palabras más probables.
- Por ejemplo, un modelo de temas analizando cientos de noticias podría encontrar un tema caracterizado por palabras {gobierno, elecciones, diputado, ley} que interpretaríamos como “Política”, y otro tema con {fútbol, campeonato, gol, entrenador} que claramente es “Deportes”.
- Un documento, a su vez, puede mezclarse de varios temas (por ej., 70% tema A, 30% tema B). El algoritmo más conocido es **LDA (Latent Dirichlet Allocation)**, que genera temas y distribuciones de temas por documento de forma probabilística. T

Conceptos básicos

LDA (Latent Dirichlet Allocation) y **STM (Structural Topic Model)** son dos técnicas de modelado de temas ampliamente utilizadas, con enfoques relacionados pero distintas capacidades.

Por ejemplo, STM podría descubrir un tema "Pandemia" y además revelar que ese tema es más frecuente en documentos del año 2020 que en los de 2018, si se incorpora *año* como covariable. En resumen, LDA da una visión estática de temas, mientras STM da una visión estructurada: temas + relación con propiedades externas de los textos.

Ejemplo: LDA es el modelo clásico de tópicos: asume que cada documento es una mezcla de T temas, y que cada palabra del documento fue generada eligiendo primero un tema y luego una palabra de la distribución de ese tema. Tras entrenar, obtienes tanto los temas (listas de palabras con sus probabilidades) como, para cada documento, el porcentaje de cada tema en él. Con STM, podrías ver que temas son más prevalentes en medicina vs. en física, porque incluirías "disciplina" como covariable que afecta la proporción de temas en cada artículo. *Ilustración sugerida:* Un gráfico comparativo: en LDA cada documento tiene una distribución de temas independiente, en STM hay gráficas que muestran influencia de una variable (e.g. año) en la proporción de cierto tema a lo largo del tiempo.

STM, en cambio, es una extensión de LDA que permite incorporar metadatos o covariables de los documentos en el proceso. Es decir, en STM uno puede modelar no solo cuáles son los temas, sino también cómo la prevalencia de cada tema en un documento puede depender de atributos como la fecha, el autor, la fuente, la longitud del documento, etc.

Conceptos básicos

Por ejemplo, STM podría descubrir un tema “Pandemia” y además revelar que ese tema es más frecuente en documentos del año 2020 que en los de 2018, si se incorpora *año* como covariable. En resumen, LDA da una visión estática de temas, mientras STM da una visión estructurada: temas + relación con propiedades externas de los textos. *Ejemplo:* Supongamos un conjunto de artículos científicos con metadato de disciplina (medicina, biología, física...). LDA te encontrará X temas generales. Con STM, podrías ver qué temas son más prevalentes en medicina vs. en física, porque incluirías “disciplina” como covariable que afecta la proporción de temas en cada artículo. *Ilustración sugerida:* Un gráfico comparativo: en LDA cada documento tiene una distribución de temas independiente, en STM hay gráficas que muestran influencia de una variable (e.g. año) en la proporción de cierto tema a lo largo del tiempo.